

Applied Linear Algebra

NumPy and PyTorch

Manipal School of Information Sciences
Manipal

Friday, 24 April 2026

Try the following in a Python shell

```
1      import numpy as np
2
3      a = np.array([1, 2, 3])
4
5      assert(a.dtype == np.int64)
6
7      # what is the shape of 'a'?
8      assert(a.shape == ?)
9
```

```
1  import numpy as np
2
3  a = np.array([1, 2, 3])
4  assert(a.shape == (3,))
5
6  assert(a.shape != (3)) # why?
7  assert(a.shape != 3) # why so?
8
9  ## what's the type of a.shape?
10 assert(isinstance(a.shape, ?))
```

```
1  import numpy as np
2
3  a = np.array([1, 2, 3])
4  assert(a.shape == (3,))
5
6  assert(a.shape != (3)) # why?
7  assert(a.shape != 3) # why so?
8
9  ## what's the type of a.shape?
10 assert(isinstance(a.shape, ?))
```

Array of Zeroes

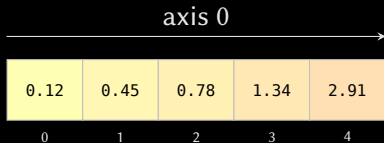
NumPy

```
1  import numpy as np
2
3  a = np.zeros(4)
4  assert a == [0, 0, 0, 0] # what's up with is?
5
6  # 'array_equal'
7  assert np.array_equal(a, [0, 0, 0, 0])
8
9  # explain this!
10 assert (a == np.array([0, 0, 0, 0])).all()
11
12 # what's the type of a.shape?
13 assert(isinstance(a.shape, ?))
```

1-Dimensional ndarray

NumPy

```
1 import numpy as np
2
3 a = np.array([0.12, 0.45, 0.78, 1.34, 2.91])
4 assert(a.shape == (5,))
5 assert(a.ndim == 1)
6 assert(a.size == 5)
```



2-Dimensional ndarray

NumPy

```
1  # A 3x5 2D ndarray/tensor
2  a = np.array([
3      [0.12, 0.45, 0.78, 1.34, 2.91],
4      [0.23, 0.56, 0.89, 1.45, 3.02],
5      [0.34, 0.67, 0.90, 1.56, 3.13],
6  ])
7  assert(a.shape == (3, 5))
8  assert(a.ndim == 2)
```

A diagram illustrating a 2D NumPy array. The array is represented as a 3x5 grid of cells. The horizontal axis is labeled 'axis 1' with an arrow pointing to the right, and the vertical axis is labeled 'axis 0' with an arrow pointing downwards. The columns are indexed 0 to 4, and the rows are indexed 0 to 2. The values in the cells are: Row 0: [0.12, 0.45, 0.78, 1.34, 2.91], Row 1: [0.23, 0.56, 0.89, 1.45, 3.02], Row 2: [0.34, 0.67, 0.90, 1.56, 3.13].

		axis 1 →				
		0	1	2	3	4
axis 0 ↓	0	0.12	0.45	0.78	1.34	2.91
	1	0.23	0.56	0.89	1.45	3.02
	2	0.34	0.67	0.90	1.56	3.13

3-Dimensional ndarray

NumPy

```
1 import numpy as np
2
3 # A 3x3x4 3D tensor: shape (planes, rows, cols)
4 a = np.random.rand(3, 3, 4)
5 assert(a.shape == (3, 3, 4))
6 assert(a.ndim == 3)
```

